

PRÁCTICA GUIADA – ETAPA 3

Proyecto “Servicios Técnicos”

Creación del Frontend de Login con React

React + JavaScript + CSS + API Flask

1. Propósito

En esta etapa se desarrollará la interfaz gráfica del inicio de sesión utilizando React, JavaScript y CSS. El formulario solicitará únicamente username y contraseña y enviará esos datos a la API Flask creada en la Etapa 2 mediante una petición HTTP POST. El alumno comprobará el flujo completo Frontend → API → MySQL → API → Frontend.

2. Resultado esperado

```
┌──────────────────────────────────┐
│       SERVICIOS TÉCNICOS         │
│                                  │
│ Username: [________________]     │
│ Contraseña: [______________]     │
│                                  │
│          [ INGRESAR ]            │
│                                  │
│ Mensaje de respuesta             │
└──────────────────────────────────┘
              │
              │ POST + JSON
              ▼
http://127.0.0.1:5000/api/login
```

3. Requisitos previos

- Etapa 1 terminada: Python y conexión con MySQL funcionando.
- Etapa 2 terminada: POST /api/login probado con Thunder Client.
- Node.js y npm instalados.
- Visual Studio Code instalado.
- El backend Flask disponible en http://127.0.0.1:5000.



4. Verificar Node.js y npm

Abre una terminal y ejecuta:

```
node --version
npm --version
```

Deben mostrar números de versión. Si alguno no es reconocido, instala Node.js antes de continuar.

5. Iniciar el backend

Abre una terminal para el backend:

```
cd backend
venv\Scripts\activate
python -m app.main
```

Debe aparecer:

```
Running on http://127.0.0.1:5000
```

Mantén esta terminal abierta.

6. Crear el proyecto React

Abre una segunda terminal y ubícate en la carpeta raíz del proyecto:

```
cd Practica_servicios_tecnicos
```

Crea el frontend con Vite:

```
npm create vite@latest frontend -- --template react
```

Cuando finalice:

```
cd frontend
npm install
```

7. Ejecutar React

```
npm run dev
```

La terminal mostrará una dirección local, normalmente:

```
http://localhost:5173/
```

Abre esa dirección en el navegador.



8. Estructura inicial del proyecto

```
Practica_servicios_tecnicos/
├── backend/
│   └── app/
├── database/
└── frontend/
    ├── src/
    │   ├── App.jsx
    │   ├── App.css
    │   └── main.jsx
    ├── index.html
    ├── package.json
    └── ...
```

9. Crear el componente Login

Abre:

```
frontend/src/App.jsx
```

Reemplaza su contenido por:

```
import { useState } from "react";
import "./App.css";

function App() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [mensaje, setMensaje] = useState("");
      const [exito, setExito] = useState(false);

  const iniciarSesion = async (e) => {
    e.preventDefault();
    setMensaje("");

    if (!username || !password) {
      setMensaje("Username y contraseña son obligatorios");
      setExito(false);
      return;
    }

    try {
      const respuesta = await fetch("http://127.0.0.1:5000/api/login", {
        method: "POST",
        headers: {
```




```
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        username: username,
        password: password
      })
    });

    const datos = await respuesta.json();
    setMensaje(datos.message);
    setExito(datos.success);
  } catch (error) {
    setMensaje("No se pudo conectar con el servidor");
    setExito(false);
  }
};

return (
  <div className="contenedor">
    <div className="login">
      <h1>Servicios Técnicos</h1>
      <h2>Iniciar sesión</h2>

      <form onSubmit={iniciarSesion}>
        <label>Username</label>
        <input
          type="text"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          placeholder="Ingrese su username"
        />

        <label>Contraseña</label>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          placeholder="Ingrese su contraseña"
        />

        <button type="submit">Ingresar</button>
      </form>

      {mensaje && (
        <p className={exito ? "mensaje exito" : "mensaje error"}>
          {mensaje}
```




```
          </p>
        )}
      </div>
    </div>
  );
}

export default App;
```

10. Crear los estilos CSS

Abre:

```
frontend/src/App.css
```

Reemplaza el contenido por:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f2f2f2;
}

.contenedor {
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

.login {
  width: 360px;
  padding: 30px;
  background: white;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.15);
}

.login h1,
.login h2 {
  text-align: center;
}
```




```
.login h1 {
  margin-bottom: 5px;
}

.login h2 {
  margin-top: 0;
  font-size: 18px;
  font-weight: normal;
}

.login label {
  display: block;
  margin-top: 15px;
  margin-bottom: 5px;
  font-weight: bold;
}

.login input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
}

.login button {
  width: 100%;
  margin-top: 20px;
  padding: 11px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
}

.mensaje {
  text-align: center;
  margin-top: 18px;
  font-weight: bold;
}

.exito {
  color: green;
}

.error {
```




```
  color: red;
}
```

11. Verificar main.jsx

No es necesario modificar main.jsx si conserva la importación de App.css. Debe tener una estructura similar a:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Si el archivo importa un CSS que ya no utilizas, puedes conservarlo o quitarlo según corresponda.

12. Comprender el fetch()

La comunicación con Flask se realiza mediante:

```
const respuesta = await fetch("http://127.0.0.1:5000/api/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    username: username,
    password: password
  })
});
```

Esta instrucción realiza desde React la misma operación que anteriormente hicimos manualmente con Thunder Client.

13. Probar el frontend

1. Asegúrate de que Flask esté ejecutándose en el puerto 5000.
2. Asegúrate de que React esté ejecutándose con npm run dev.
3. Abre la dirección que proporciona Vite, normalmente http://localhost:5173/.
4. Debe aparecer el formulario de Servicios Técnicos.



5. Escribe el usuario de prueba.
6. Escribe la contraseña de prueba.
7. Haz clic en Ingresar.

14. Prueba de login correcto

Utiliza las credenciales que existen en MySQL. Por ejemplo:

```
Username: admin
Contraseña: 1234
```

Resultado esperado en pantalla:

```
Inicio de sesión correcto
```

La consola de Flask debe registrar:

```
POST /api/login ... 200
```

15. Prueba de contraseña incorrecta

```
Username: admin
Contraseña: 99999
```

Resultado esperado:

```
Usuario o contraseña incorrectos
```

Flask debe responder con HTTP 401.

16. Prueba de campos vacíos

Deja ambos campos vacíos y presiona Ingresar.

Resultado esperado:

```
Username y contraseña son obligatorios
```

En este caso React realiza la validación antes de enviar la petición.

17. Prueba de servidor apagado

Detén temporalmente Flask con Ctrl + C y vuelve a presionar Ingresar.

Resultado esperado:

```
No se pudo conectar con el servidor
```



Vuelve a iniciar Flask para continuar.

18. Posible problema de CORS

Si el navegador muestra un error relacionado con CORS, comprueba que el backend tenga instalado Flask-CORS y que main.py contenga:

```
from flask_cors import CORS

app = Flask(__name__)
CORS(app)
```

Después reinicia Flask.

19. Flujo completo del sistema

```
┌──────────────────────┐
│ React                │
│ Formulario Login     │
└──────────┬───────────┘
           │ POST + JSON
           ▼
┌──────────────────────┐
│ Flask                │
│ /api/login           │
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│ conexion.py          │
└──────────┬───────────┘
           │ SQL
           ▼
┌──────────────────────┐
│ MySQL                │
│ tecnico.usuario      │
└──────────┬───────────┘
           │ resultado
           ▼
      JSON → React
```



20. Solución de problemas

Problema	Solución
npm no se reconoce	Instalar Node.js y reiniciar VS Code.
Failed to fetch	Comprobar que Flask esté ejecutándose en el puerto 5000.
CORS error	Comprobar Flask-CORS y CORS(app).
404 /api/login	Comprobar la URL y el Blueprint de login.
401	Las credenciales no coinciden con MySQL.
500	Revisar la terminal de Flask y la conexión a MySQL.
Pantalla en blanco	Revisar la consola del navegador y App.jsx.

21. Evidencias

- Captura de npm run dev mostrando React funcionando.
- Captura del formulario de login.
- Captura del login correcto.
- Captura del login incorrecto.
- Captura de la terminal Flask mostrando POST /api/login.
- Captura de la estructura frontend/src.
- Captura del repositorio GitHub actualizado.

22. Preguntas de análisis

8. ¿Qué función cumple React en el proyecto?

Respuesta: _____

_____

9. ¿Qué función cumple fetch()?

Respuesta: _____

_____

10. ¿Por qué se utiliza JSON.stringify()?

Respuesta: _____

_____



11. ¿Qué diferencia existe entre Thunder Client y React en esta etapa?

Respuesta: _____

_____

12. ¿Por qué React no se conecta directamente a MySQL?

Respuesta: _____

_____

13. ¿Qué función cumple CORS?

Respuesta: _____

_____

14. ¿Qué ocurre cuando Flask devuelve HTTP 401?

Respuesta: _____

_____

23. Registrar la Etapa 3 en GitHub

Detén React con Ctrl + C cuando termines. Desde la raíz del proyecto ejecuta:

```
git status
git add .
git commit -m "Implementa frontend de login con React"
git push
```

Verifica que el commit aparezca en GitHub.



24. Criterios de aceptación

- ☐ React inicia correctamente.
- ☐ El formulario solicita únicamente username y contraseña.
- ☐ Los campos funcionan con useState.
- ☐ El formulario utiliza onSubmit.
- ☐ Se envía POST /api/login.
- ☐ El Content-Type es application/json.
- ☐ Las credenciales correctas muestran mensaje de éxito.
- ☐ Las credenciales incorrectas muestran mensaje de error.
- ☐ Los campos vacíos se validan.
- ☐ El frontend se comunica con Flask.
- ☐ El proyecto fue actualizado en GitHub.